codigofacilito

Clase 12 · Bootcamp AI Engineer

Memoria Persistente y Human-in-the-Loop

Checkpointing, persistencia de estado y aprobaciones humanas

Instructor: Ramsés Camas

¿Qué veremos hoy?

01 Recapitulación: nuestro sistema multiagente

02 El problema: agentes sin memoria y sin supervisión

03 Persistencia y checkpointing en LangGraph

04 Memoria de conversación entre sesiones

05 Human-in-the-Loop: la función interrupt()

06 Patrones HITL: aprobar, editar, rechazar, escalar

07 Proyecto: memory/store.py y agents/hitl.py

Recapitulación: nuestro DocOps Agent

Funciona, pero tiene dos problemas graves para producción

Lo que construimos (Clase 11)

- Planner → Retriever → Executor → Verifier
- Bucle de auto-corrección (max 3 iteraciones)
- Fallback a modelo de respaldo

Problema 1: Sin memoria

Cada invocación empieza de cero. Si el usuario pregunta "¿y qué más dice ese documento?" el agente no tiene contexto previo.

Problema 2: Sin supervisión humana

El agente ejecuta acciones automáticamente. Si el Executor genera algo incorrecto que el Verifier acepta, no hay forma de que un humano intervenga. En tareas críticas (borrar datos, enviar emails, aprobar gastos), esto es inaceptable.

Persistencia + HITL = agentes confiables

Las dos piezas que faltan para producción

Persistencia

- Guarda estado completo tras cada paso
- Si el proceso muere, retoma donde quedó
- Conversaciones se recuerdan entre sesiones

Como "guardar partida" en un videojuego: posición, inventario, progreso — todo

Human-in-the-Loop

- Humano revisa y aprueba acciones
- Agente se pausa, espera decisión
- Aprobar, editar, rechazar o escalar

Retoma exactamente donde se pausó — horas o días después, en otra máquina

Relación clave: HITL depende de persistencia. Sin checkpointing, no puedes pausar y reanudar.

Sin checkpointer, cada invocación es amnésica

```
from agents.multi_agent_graph import docops_agent

# Primera consulta
result1 = docops_agent.invoke({
    "messages": [HumanMessage(content="¿Cuál es la política de reembolso?")],
    "plan": "", "search_results": "", "draft": "",
    "feedback": "", "quality_score": 0.0, "iteration": 0,
})
print(result1["draft"])  # Respuesta correcta sobre reembolsos

# Segunda consulta — NO recuerda la primera
result2 = docops_agent.invoke({
    "messages": [HumanMessage(content="¿Y para clientes premium?")],
    "plan": "", "search_results": "", "draft": "",
    "feedback": "", "quality_score": 0.0, "iteration": 0,
})
# El agente NO sabe que "premium" se refiere a reembolsos
# Porque el estado se reinicia en cada invoke()
```

El estado se pierde entre llamadas. Necesitamos un checkpointer + thread_id.

Checkpointing: guardar el estado en cada paso

Un snapshot completo del grafo después de cada nodo

¿Qué se guarda?

- Todos los valores del estado
- Qué nodo ejecutó y cuál sigue
- Configuración del thread
- Metadatos de auditoría

Tipos de checkpointer

`MemorySaver` → RAM (desarrollo)

`SqliteSaver` → archivo .db (demo)

`PostgresSaver` → PostgreSQL (producción)

La línea más importante

El grafo se compila con un checkpointer, y la lógica de los nodos no cambia absolutamente nada.

thread_id: la clave de todo el sistema

Un identificador único que le dice a LangGraph qué conversación retomar

¿Cómo se usa?

1. Se pasa en el config al invocar el grafo
2. LangGraph busca si existe un checkpoint para ese thread
3. Si existe: carga el estado y continúa
4. Si no existe: inicia conversación nueva

Ejemplos de thread_id

`"usuario-123-sesion-1"`
→ chat por usuario

`"ticket-2024-0042"`
→ ticket de soporte

`"workflow-reembolso-7"`
→ tarea específica

Analogía: el thread_id es como el slot de partida guardada. Cada slot tiene su progreso independiente.

memory/store.py — Persistencia con MemorySaver

```
"""memory/store.py — Checkpointing para DocOps Agent"""
from langgraph.checkpoint.memory import MemorySaver

# En desarrollo: checkpointer en memoria
checkpointer = MemorySaver()

# En producción:
# from langgraph.checkpoint.sqlite import SqliteSaver
# checkpointer = SqliteSaver.from_conn_string("docops.db")
#
# from langgraph.checkpoint.postgres import PostgresSaver
```

```
# En multi_agent_graph.py — solo cambia UNA línea
from memory.store import checkpointer
# Antes:  docops_agent = workflow.compile()
# Ahora:
docops_agent = workflow.compile(checkpointer=checkpointer)
# Invocar CON thread_id
config = {"configurable": {"thread_id": "usuario-42"}}
result = docops_agent.invoke(initial_state, config)
```

Una sola línea transforma un agente amnésico en uno con memoria persistente.

Memoria de conversación entre sesiones

messages se acumula automáticamente entre invocaciones del mismo thread_id

1. Usuario: "¿Política de reembolso?"
2. Agente procesa: planner → retriever → executor → verifier
3. Checkpoint guarda estado completo (incluye messages)
4. Usuario: "¿Y para clientes premium?"
5. LangGraph carga el checkpoint → agente ve AMBOS mensajes
6. Planner genera plan con contexto de toda la conversación

¿Qué cambia en el código? Nada. La magia está en el checkpointer + thread_id, no en los nodos.

Inspeccionar, modificar y auditar el estado

Tres operaciones fundamentales sobre el estado persistido

`get_state(config)`

Inspeccionar: ver snapshot actual, qué nodo ejecutó, cuál sigue, todos los valores.

`update_state(config, updates)`

Modificar: corregir valores sin re-ejecutar el grafo. El humano edita el draft directamente.

`get_state_history(config)`

Auditar: todos los snapshots anteriores. Puedes "rebobinar" a cualquier punto.

Para compliance: responde a "¿por qué el agente tomó esta decisión?" — obligatorio en finanzas, salud, legal.

Operaciones sobre el estado persistido

```
config = {"configurable": {"thread_id": "usuario-42"}}

# ─── INSPECCIONAR estado actual ───
snapshot = docops_agent.get_state(config)
print(f"Siguiente nodo: {snapshot.next}")
print(f"Plan actual: {snapshot.values['plan'][:200]}")
print(f"Quality score: {snapshot.values['quality_score']}")

# ─── MODIFICAR estado (corrección humana) ───
docops_agent.update_state(
    config,
    {"draft": "Respuesta corregida por un humano experto."},
    as_node="executor"  # Simula que el executor lo escribió
)
```

```
# ─── HISTORIAL completo (auditoría) ───
for i, state in
enumerate(docops_agent.get_state_history(c
onfig)):
    node = state.next[0] if state.next
else "END"
    print(f"Paso {i}: -> {node} | "

f"score={state.values.get('quality_score',
'N/A')}")

# ─── REBOBINAR a un snapshot anterior
───
past =
list(docops_agent.get_state_history(config
))[3].config
docops_agent.update_state(past, {})  #
Restaura ese punto
```

get_state, update_state y get_state_history — las 3 herramientas clave de supervisión.

codigofacilito

Human-in-the-Loop

Porque los agentes no son perfectos — y no necesitan serlo

HITL: humanos y agentes trabajando juntos

No es manejo de errores — es una capa de middleware para supervisión

Aprobar / Rechazar

"Voy a enviar este email. ¿Apruebas?"

Editar

"Aquí está mi borrador. Puedes editarlo."

Revisar tool calls

"Quiero ejecutar DELETE. ¿Confirmas?"

Pedir información

"¿A cuál de los 3 documentos te refieres?"

¿Cuándo? Acciones irreversibles, impacto financiero/legal, baja confianza, ramp-up de sistema nuevo.

interrupt() — el input() de producción

Pausa el grafo, guarda estado, espera al humano, reanuda

1. Dentro de un nodo, llamas interrupt("mensaje")
2. LangGraph pausa ejecución y guarda estado en checkpointer
3. El humano revisa la información y toma una decisión
4. Se reanuda con Command(resume=respuesta_humana)
5. El nodo se re-ejecuta; interrupt() retorna la respuesta

Requisitos: checkpointer configurado + thread_id en config + NUNCA envolver interrupt() en try/except

Patrón básico: aprobación humana

```
from langgraph.types import interrupt, Command

def executor_with_approval(state: DocOpsState):
    """Executor que pide aprobacion antes de entregar."""
    draft = llm.invoke([
        SystemMessage(content="Genera una respuesta precisa."),
        HumanMessage(content=f"Plan: {state['plan']}\n"
                     f"Contexto: {state['search_results']}")
    ]).content

    # ── PUNTO DE INTERRUPCION ──
    decision = interrupt({
        "message": "Revisa esta respuesta antes de entregarla.",
        "draft": draft,
        "action": "approve_or_edit",
    })

    # Se ejecuta DESPUES de que el humano
responde
    if decision.get("approved"):
        return {"draft":
decision.get("edited_draft", draft)}
    return {"draft": "[RECHAZADO]",
"quality_score": 0.0}
```

Todo lo que está ANTES de interrupt() se re-ejecuta al reanudar. Las operaciones deben ser idempotentes.

Reanudando el grafo con Command(resume)

El flujo completo: invocar → pausar → decidir → reanudar

Paso 1: Invocación inicial

result = graph.invoke(state, config)

El grafo ejecuta hasta llegar al interrupt()

result contiene la info de la interrupción

Paso 2: Inspeccionar la interrupción

Frontend/CLI muestra al humano los datos

El humano toma su decisión (aprobar, editar, rechazar)

Paso 3: Reanudar

graph.invoke(Command(resume={...}), config)

El nodo se re-ejecuta desde el inicio

interrupt() retorna la respuesta del humano

El valor de Command(resume=...) se convierte en el retorno de interrupt() dentro del nodo.

No todo necesita aprobación humana

Interrumpir solo cuando el riesgo justifica la intervención

Bajo	**Auto-aprobado** Consultas de lectura, búsquedas
Medio	**Supervisión pasiva** Se loggea para auditoría async
Alto	**Aprobación requerida** Escrituras, envío emails, cambios config
Crítico	**Doble aprobación** Borrar datos, transacciones financieras

El nivel se determina por: tipo de herramienta, monto, quality_score, o si es primera ejecución.

agents/hitl.py — Gate humano condicional

```python
"""agents/hitl.py — Human-in-the-Loop para DocOps Agent"""
from langgraph.types import interrupt

def assess_risk(state: DocOpsState) -> str:
    """Evalua el nivel de riesgo de la accion actual."""
    draft = state.get("draft", "")
    score = state.get("quality_score", 0.0)
    if score < 0.6:
        return "high"
    risky = ["eliminar", "borrar", "enviar", "delete", "send"]
    if any(kw in draft.lower() for kw in risky):
        return "high"
    return "low"
```

```python
def human_gate(state: DocOpsState) -> dict:
    """Gate que decide si pausar para aprobacion
humana."""
    risk = assess_risk(state)

    if risk in ("high", "critical"):
        decision = interrupt({
            "risk_level": risk,
            "message": f"Accion de riesgo {risk}
detectada.",
            "draft_preview": state["draft"][:500],
            "action": "approve_edit_reject",
        })
        if not decision.get("approved"):
            return {"draft": f"[RECHAZADO:
{decision.get('reason')}]",
                    "quality_score": 0.0}
        if decision.get("edited_draft"):
            return {"draft": decision["edited_draft"]}

    return state  # Riesgo bajo: continuar sin
interrupcion
```

El gate se inserta como nodo entre verifier y END. Solo pausa cuando hay riesgo.

Regla de oro: idempotencia antes de interrupt()

Al reanudar, el nodo se re-ejecuta COMPLETO desde el inicio

Problemático si antes de interrupt():

- Llamadas a LLM → doble cobro
- Insert en BD → datos duplicados
- Envío de emails → usuario recibe dos
- Cualquier operación con side effects

Soluciones:

- Cachear resultados con ID único
- Usar upsert en vez de insert
- Verificar si email ya fue enviado
- **Separar lógica costosa en nodo aparte (antes del interrupt)**

Mejor práctica: poner la lógica costosa en un nodo sin interrupt, y la aprobación en un nodo dedicado.

Auditoría: quién hizo qué, cuándo y por qué

Registro completo de cada decisión del agente y cada intervención humana

Por cada paso del agente

- Timestamp, nodo, duración
- Input recibido, output producido
- Modelo LLM, tokens, costo
- Herramientas invocadas y resultados

Por cada intervención humana

- Quién aprobó/rechazó (user_id)
- Qué se presentó y qué decidió
- Antes y después si editó el draft
- Razón del rechazo (si aplica)

Responde: "¿Quién autorizó que el agente enviara este email?" — obligatorio en industrias reguladas.

Logging estructurado para auditoría

```
import json, logging
from datetime import datetime, timezone

logger = logging.getLogger("docops.audit")

def log_agent_action(node: str, state: dict, result: dict):
    """Registra cada accion del agente."""
    entry = {
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "node": node,
        "iteration": state.get("iteration", 0),
        "quality_score": state.get("quality_score"),
        "draft_length": len(state.get("draft", "")),
    }
    logger.info(json.dumps(entry))
```

```
def log_human_decision(thread_id: str, decision:
dict,
                       user_id: str):
    """Registra cada intervencion humana."""
    entry = {
        "timestamp":
datetime.now(timezone.utc).isoformat(),
        "thread_id": thread_id,
        "user_id": user_id,
        "approved": decision.get("approved"),
        "edited": "edited_draft" in decision,
        "reason": decision.get("reason", ""),
        "risk_level": decision.get("risk_level"),
    }
    logger.info(json.dumps(entry))
```

En producción, estos logs van a un sistema centralizado (ELK, Datadog) — lo veremos en Clase 16.

Dónde insertar el gate humano en DocOps Agent

Entre verifier y END — evalúa riesgo antes de entregar

Clase 11: START → Planner → Retriever → Executor → Verifier → END

Clase 12: START → Planner → Retriever → Executor → Verifier → Gate → END

Flujo completo con todas las piezas:

1. **Planner** planifica (auto)
2. **Retriever** busca (auto)
3. **Executor** genera (auto + fallback)
4. **Verifier** evalúa calidad (bucle si necesario)
5. **Human Gate** evalúa riesgo
6. **Si riesgo alto:** interrupt() → humano revisa → Command(resume) → continúa
7. Respuesta entregada al usuario

De MemorySaver a PostgresSaver

Cambiar de desarrollo a producción requiere cambiar UNA línea

Desarrollo	`MemorySaver`	✓ Cero config, ultra rápido ✗ Se pierde si el proceso muere
Demo / MVP	`SqliteSaver`	✓ Sobrevive reinicios, archivo local ✗ Un solo proceso puede acceder
Producción	`PostgresSaver`	✓ Multi-proceso, escala, durable ✗ Requiere PostgreSQL corriendo

El resto del código del grafo no cambia absolutamente nada. Decisión de diseño intencional de LangGraph.

DocOps Agent v2 — Con memoria y HITL

```python
"""Actualizacion de multi_agent_graph.py para Clase 12"""
from memory.store import checkpointer
from agents.hitl import human_gate

workflow = StateGraph(DocOpsState)

# Mismos 4 agentes de Clase 11
workflow.add_node("planner", planner_agent)
workflow.add_node("retriever", retriever_agent)
workflow.add_node("executor", executor_agent)
workflow.add_node("verifier", verifier_agent)
workflow.add_node("human_gate", human_gate)   # NUEVO

# Flujo principal
workflow.add_edge(START, "planner")
workflow.add_edge("planner", "retriever")
workflow.add_edge("retriever", "executor")
workflow.add_edge("executor", "verifier")

# Bucle de calidad (sin cambios)
workflow.add_conditional_edges("verifier", should_revise, {
    "accept": "human_gate",   # Antes iba a END
    "revise": "executor",
})
workflow.add_edge("human_gate", END)

# Compilar CON checkpointer
docops_agent = workflow.compile(checkpointer=checkpointer)

# Invocar con thread_id
config = {"configurable": {"thread_id":
"ticket-2024-0042"}}
result = docops_agent.invoke(initial_state, config)

# Si se interrumpio, reanudar con decision humana
from langgraph.types import Command
docops_agent.invoke(Command(resume={"approved": True}),
config)
```

Recapitulación y siguiente paso

Lo que construimos hoy

- Checkpointing (MemorySaver)
- Memoria de conversación (thread_id)
- Inspección del estado (get/update/history)
- HITL con interrupt() + Command(resume)
- Interrupciones condicionales por riesgo
- Logging estructurado para auditoría

Clase 13: MCP

- Model Context Protocol (estándar)
- MCP Server: exponer herramientas
- MCP Client: consumir herramientas

Artefactos:
`memory/store.py` + `agents/hitl.py`

Tarea

Implementar un CLI interactivo que use DocOps Agent con HITL. Cuando el agente se pause con interrupt(), el CLI muestra el draft, pide input (aprobar/editar/rechazar), y reanuda con Command(resume). Usar SqliteSaver para que la conversación persista entre ejecuciones del script.